

Triagem Automática de Peças de Vestuário

Relatório Técnico

Rodolfo Brandão
(rodolfo.brandao@souunit.com.br)

Proposta

A ideia do projeto foi pensada em cima de um marketplace, no qual os vendedores sobem fotos das peças de roupa que querem anunciar. Antes do anúncio ir ao ar, é necessário dizer em qual categoria do catálogo cada peça se encaixa. O processo habitual é manual, o trabalho é lento e caro. Mas uma vez feito por um modelo de IA, torna-se instantâneo. Contudo, nenhum modelo é sempre certo e uma peça catalogada de forma errada simplesmente não aparece para quem a procura.

O projeto parte dessa tensão: em vez de mirar em um classificador perfeito, capaz de acertar tudo, foi desenvolvido um que resolve a maior parte dos casos e sabe reconhecer quando está em dúvida. Caso a confiança do modelo não seja suficiente na resposta, a foto vai para uma fila de revisão humana. Com isso, a qualidade do que entra no catálogo sem supervisão fica bem acima da acurácia bruta do modelo, ao custo de manter parte do volume sob revisão.

Esse é um problema de visão computacional que envolve classificação de imagens em múltiplas classes: dada uma foto, dizer a qual das dez categorias do catálogo ela pertence.

Sobre os Dados

O dataset utilizado foi o **Fashion-MNIST**, no qual contém 70.000 imagens em tons de cinza, de 28x28 pixels, divididas em dez categorias de vestuário:

1. Camiseta/top
2. Calça
3. Pullover
4. Vestido
5. Casaco
6. Sandália
7. Camisa
8. Tênis

- 9. Bolsa
- 10. Bota

A divisão separa 60.000 imagens para treino e 10.000 para teste, além de possuir um balanceamento de 6.000 exemplos para cada classe no treino. Esse dataset tem o mesmo formato do MNIST de dígitos, mas o problema visual é mais difícil: distinguir alguns itens exige detalhes que 28x28 em cinza quase não carrega, e ainda assim treina em poucos minutos em um computador com configurações relativamente baixas.

Pré-processamento

Essa etapa se deu em duas partes:

- **Escala dos pixels:** de 0 à 255 para decimais entre 0 e 1. Redes neurais treinam de forma mais estável com entradas numa faixa pequena e uniforme.
- **Formato do tensor:** de 28x28 para 28x28x1. A camada convolucional espera o eixo de canais explícito; aqui ele vale 1, por ser tons de cinza (uma imagem colorida teria 3).

Dos 60.000 exemplos de treino, 10% ficaram reservados para validação durante o treinamento. As 10.000 imagens de teste só foram usadas na avaliação final.

Arquitetura

Modelo de Referência

Antes de montar a rede convolucional, uma rede densa simples foi treinada com a finalidade de achatar a imagem num vetor de 784 pixels e passar por uma única camada de 128 neurônios até a saída de 10 classes. Esse modelo existe por um motivo: ao achatar a imagem, a informação de vizinhança é descartada. Qualquer ganho da CNN é atribuível ao fato de as convoluções olharem a imagem como imagem. Sem esse tratamento, dizer que a CNN “teve bom desempenho” seria uma afirmação vazia.

Rede Convolucional

A rede principal segue a estrutura de referência vista em sala de aula, no notebook do CIFAR-10, adaptada para imagens de um canal:

Camada	Saída	Parâmetros
Entrada	28x28x1	1
Conv2D - 32 filtros 3x3 (ReLU)	28x28x32	320
MaxPooling 2x2	14x14x32	0
Conv2D - 64 filtros 3x3 (ReLU)	14x14x64	18496

MaxPooling 2x2	7x7x64	0
Flatten	3136	0
Densa - 128 neurônios (ReLU)	128	401536
Dropout - 30%	128	0
Densa - 10 neurônios (softmax)	10	1290

A lógica é a de sempre: a primeira convolução aprende padrões locais, como bordas e mudanças de tom; o *pooling* reduz a imagem pela metade e faz a rede depender menos da posição exata de cada detalhe; a segunda convolução combina esses padrões em formas maiores (o contorno de uma manga, a curva de um salto). Só então a parte densa decide a classe.

A saída é uma camada *softmax* com dez valores que somam 1, e é ela que dá ao modelo um segundo uso: além da categoria escolhida (o maior valor), o tamanho desse valor funciona como grau de confiança (é ele que aciona ou não a revisão humana).

Treinamento

As configurações do treino foram:

- **Otimizador:** Adam, com taxa de aprendizado padrão.
- **Função de Perda:** `sparse_categorical_crossentropy` - indicada quando os rótulos são números inteiros de classe em vez de vetores.
- **Lotes:** 128 imagens por passo.
- **Épocas:** até 20, com parada antecipada.
- **Parada Antecipada:** `EarlyStopping` - monitora a perda de validação, com *paciência* de 3 épocas. A rede treina enquanto melhora e, após três épocas sem melhoria, o treino para e os pesos voltam para os da melhor época.
- **GPU:** Apple Silicon M4 (Metal Performance Shaders), utilizando o backend PyTorch do Keras 3.

Detalhe prático: o Keras decide qual backend vai usar no momento em que é importado. Definir `KERAS_BACKEND` depois do `import keras` não surte efeito pois a biblioteca já se configurou. Por isso, no notebook, essa definição é a primeira célula de código.

Na execução usada como referência neste relatório, o treino da CNN parou sozinho na época 14, com os melhores pesos vindos da época 11, e levou cerca de 2 minutos no total (aproximadamente 9 segundos por época). A linha de base treinou por 5 épocas fixas, em 9 segundos.

Análise dos Resultados

Desempenho Geral

Modelo	Parâmetros	Acurácia no Teste
Rede densa (linha de base)	101770	86,7%
Rede convolucional	421642	92,2%

Uma diferença de 5,4 pontos percentuais. Dito de forma mais concreta: das 10.000 fotos de teste, a rede densa erra 1.327 e a CNN erra 783 (41% menos peças na categoria errada), ao custo de quatro vezes mais parâmetros e de 2 minutos de treino em vez de 9 segundos.

Como não há *seed* aleatória fixa, cada treino termina num ponto um pouco diferente: em três execuções completas a CNN ficou entre 91,7% e 92,2%. A variação não muda nenhuma conclusão, mas é honesto registrar que os números não se repetem exatamente.

Onde o Modelo Erra

A acurácia geral esconde o mais interessante. Olhando classe a classe:

Classe	Precisão	Revocação	F1
Calça	0,993	0,986	0,989
Sandália	0,989	0,983	0,986
Bolsa	0,987	0,980	0,983
Bota	0,979	0,965	0,972
Tênis	0,953	0,977	0,965
Vestido	0,919	0,923	0,921
Pullover	0,899	0,872	0,885
Casaco	0,867	0,892	0,879
Camiseta/Top	0,847	0,889	0,868
Camisa	0,784	0,750	0,766

O padrão faz sentido visual. As peças de formato próprio, como calça, sandália, bolsa, tênis e bota passam de 0,96 de F1: são silhuetas que não se confundem com nada. Já as peças de

tronco vivem disputando entre si, e a camisa é disparada a pior classe (0,77 de F1 e 75% de revocação: uma em cada quatro camisas é catalogada como outra coisa). Em 28x28 e sem cor, camisa, camiseta, pullover e casaco têm quase o mesmo contorno; o que os separa no mundo real são botões, gola, textura e caimento (justamente os detalhes que essa resolução descarta).

As trocas mais frequentes na matriz de confusão confirmam a leitura:

Real -> Predito	Ocorrências
Camisa -> Camiseta/Top	114
Camiseta/Top -> Camisa	78
Camisa -> Casaco	68
Pullover -> Camisa	58
Pullover -> Casaco	46
Casaco -> Camisa	42

Essas seis trocas sozinhas respondem por 406 dos 783 erros (mais da metade). Os erros não estão espalhados pelo problema: concentram-se num punhado previsível de pares. Isso é uma boa notícia para o desenho do sistema, porque erro previsível é erro que dá para tratar.

Limitações

O dataset é fácil demais para o problema que representa. As imagens do Fashion-MNIST são recortadas, centralizadas, com fundo uniforme e iluminação constante; fotos de vendedores reais chegam tortas, com a peça dobrada, em cabide ou manequim, fundo de quarto e sombra. Os 92% medem o modelo neste dataset, não a viabilidade da aplicação.

A confiança do softmax não é uma probabilidade calibrada. No teste, a confiança média foi 0,96 nos acertos e 0,73 nos erros: o modelo de fato hesita mais quando erra (é o que faz a triagem funcionar), mas 0,73 ainda é alto para uma resposta errada, e cerca de 175 erros passam do limiar de 0,90. Redes neurais são reconhecidamente mal calibradas, e tratar a saída do softmax como probabilidade real é uma simplificação que assumida neste projeto.

A acurácia trata todos os erros como iguais, quando eles não são. Catalogar uma sandália como tênis é um deslize pequeno; catalogar uma bolsa como vestido tira a peça do lugar onde alguém a procuraria. Nenhuma métrica aqui captura essa diferença de custo.

O catálogo é fechado. O modelo só conhece as dez categorias que viu: se chegar a foto de um cinto ou de um chapéu, vai responder com uma das dez, possivelmente com confiança alta.

Um catálogo real cresce, tem hierarquia (roupa -> parte de cima -> camisa social) e recebe itens que não existiam no treino.

Melhorias

Aumento de dados (*data augmentation*). Espelhar horizontalmente, deslocar e girar levemente as imagens durante o treino é uma abordagem barata, ataca diretamente o sobreajuste observado e ensina a rede a não depender do enquadramento perfeito. Seria o primeiro passo a ser dado.

Calibrar a confiança. Técnicas como *temperature scaling* ajustam a saída para que "90% de confiança" signifique de fato 90% de acerto. Como toda a aplicação depende desse número, é a melhoria de maior impacto.

Ajustes na arquitetura. Trocar o `Flatten` por *global average pooling* eliminaria a maior parte dos 400 mil parâmetros da camada densa, deixando o modelo menor sem perder capacidade de visão. Normalização em lote acelera e estabiliza o treino.

Transferência de aprendizado. Para o cenário real, com fotos coloridas e em alta resolução, o caminho não é treinar uma CNN pequena do zero, e sim partir de uma rede já treinada em milhões de imagens (como ResNet ou EfficientNet, por exemplo) e adaptá-la ao catálogo.

Fechar o ciclo. As peças enviadas para revisão são justamente os casos difíceis. Guardar a correção do revisor e usá-la para treinar novamente transforma a fila de revisão em fonte de dados: o modelo passa a melhorar exatamente onde é fraco.